Product Requirements Document (PRD)

ShopEase — Billing & POS System for Indian Shop Owners

Version: 2.0 (Final) **Product Name:** ShopEase **Target Users:** Small and mid-size shop owners across India **Primary Use Case:** Clothing shops, kirana stores, textile shops, and any retail shop in India **Document Owner:** Development Team

---

Table of Contents

---

1. Product Overview

What Are We Building?

ShopEase is a web-based billing and point-of-sale (POS) system designed specifically for Indian shop owners. It replaces manual handwritten billing with a fast digital system that works on any mobile, tablet, or desktop browser.

The Problem It Solves

Indian shop owners today face these daily problems:

- Billing is done manually on paper — slow and error-prone
- No record of what was sold, how much profit was made
- No way to track stock — they don't know when items run out

- Tax (GST) calculation is done manually — mistakes happen
- No summary of daily or monthly business performance
- Customers negotiate prices but there is no clean way to record the final agreed price

The Solution

A shop owner logs in, enters the customer name, adds products by scanning a barcode or searching by name or using voice, applies any discount or negotiated price, selects the payment mode, and the system generates and prints a bill — all in under 60 seconds.

At 9 PM every day and on the 1st of every month, the system automatically emails the shop owner a full report of sales, profit, stock status, and transactions.

---

2. Target Users and Goals

Primary User — Shop Owner (e.g., Ravi, owner of Ravi's Clothing Store, Bangalore)

Goal	How ShopEase Solves It
Generate bills fast	Barcode scan, voice, or search adds items in seconds
Apply negotiated price	Edit any item price before billing
Calculate GST automatically	System applies the correct GST rate per product category
Print bill at the store	Connects to a thermal printer — customer takes physical bill
Know today's profit at end of day	Daily email at 9 PM with full summary
Know monthly business health	Monthly email on 1st with full report + PDF attachment
Track stock levels	Dashboard shows stock; email alerts when items are low

Customer (e.g., Arjun Kumar, walking into the shop)

The customer does not log in or create an account. They interact only by:

- Giving their name (required for the bill)
- Optionally giving their phone number (only if they choose to — privacy respected)
- Paying by cash, UPI, PhonePe, Google Pay, or card
- Receiving a printed bill from the thermal printer at the store

---

3. Core Features

Feature 1 — Login and Authentication

Who logs in: Shop owner only. Customers do not log in.

Login Method:

- Mobile number + OTP (preferred — most Indian shop owners don't remember passwords)
- Alternatively: Email + password

After Login:

- The website shows the shop name prominently at the top
- Dashboard loads with today's sales summary
- Session stays active for 8 hours (so Ravi doesn't re-login during working hours)

Scalability Requirement: The login system must handle 1,00,000 (one lakh) concurrent logins without slowing down or crashing. This is achieved using Redis for session caching and a load balancer across multiple backend servers. (Explained in detail in Section 4.)

Feature 2 — Dashboard (Home Screen After Login)

What the owner sees immediately after login:

- Shop name and logo at the top
- Today's total sales in ₹
- Number of bills generated today
- Today's profit (estimated based on cost price vs selling price)
- Quick action buttons: New Bill, View Products, Sales History
- Low stock alerts — products below threshold shown in red

Feature 3 — New Bill — Customer Details

When the shop owner taps "New Bill", the first screen asks for customer information.

Field	Required?	Notes
Customer name	YES — required	Used on the printed bill
Customer phone number	NO — fully optional	Many Indian customers do not share phone numbers due to privacy. This field is offered but never forced. If given, it can be used later to send a WhatsApp bill optionally.

Important design note: The phone number field must have a clear label saying "Optional — customer's choice". No asterisk, no red border, no warning if left blank. The bill works perfectly with just the name.

Feature 4 — Product Selection (Three Methods)

After customer details, the owner adds products to the bill using any of these three methods:

Method 1 — Barcode Scan

- Camera opens on mobile or tablet
- Owner points camera at the barcode on the product tag
- System identifies the product: name, MRP, GST rate auto-fill
- Product is added to the cart instantly
- Works even with slow internet — product list is cached on the device

Method 2 — Voice Input

- Owner taps the microphone icon
- Speaks the product name in English or Hindi (e.g., "blue shirt" or "नीली शर्ट")
- System searches and shows matching products
- Owner taps the correct one to add it

Method 3 — Manual Search

- Search bar at the top
- Type product name, category, or code
- Results appear instantly as you type
- Tap to add to cart

Adding quantity: Each product in the cart has + and − buttons to change quantity. Default is 1.

Feature 5 — Billing Screen

After all products are added, the billing screen shows the full order and allows the owner to apply discounts, negotiated prices, and coupons.

5.1 Order Summary

Column	Description
Product name	Name and size/variant
MRP	Original price printed on the product
Selling price	What the customer will actually pay
Quantity	Number of units
Item total	Selling price × quantity

5.2 Price Negotiation

A core feature for Indian retail shops where price bargaining is common.

- Every product line in the cart has an "Edit price" button
- The owner taps it and types the agreed price
- Example: Jeans MRP is ₹1,800. Customer negotiates ₹1,500. Owner types 1500.
- The system shows both the MRP and the new price side by side
- Discount amount (₹300) and discount percentage (16.7%) are auto-calculated
- GST is recalculated on the new price, not on MRP

5.3 Coupon and Offer System

Offer Type	Example
Flat % off entire bill	10% off on total
Flat ₹ amount off	₹200 off on purchase above ₹1,000
Buy 1 Get 1 Free (same item)	Buy one shirt, get one shirt free
Combo offer	Buy a shirt, get a pant at 50% off
Minimum purchase offer	Spend ₹3,000, get ₹500 off

The owner enters the coupon code in the billing screen. System validates it and applies the discount automatically. If a customer does not have a coupon, this field is simply left blank.

5.4 GST Calculation

GST is applied automatically based on the product category registered by the owner:

Category	GST Rate
Clothing below ₹1,000	0%
Clothing above ₹1,000	5%
Footwear below ₹500	0%
Footwear above ₹500	18%
Electronics	18%
Grocery / food	0% to 5%

The owner sets the GST rate when adding a product. The system then applies it automatically on every bill — no manual calculation ever needed.

5.5 Final Bill Breakdown

```
Product 1: Blue cotton shirt (L) × 1        ₹1,200
Product 2: Black slim jeans (32) × 1        ₹1,500  [MRP: ₹1,800]

Subtotal                                    ₹2,700
Discount (negotiated on jeans)               - ₹300
GST @ 5% on ₹2,700                          + ₹135
──────────────────────────────────────────────────
Total Payable                               ₹2,835
```

Feature 6 — Payment

The owner selects how the customer is paying:

Mode	How It Works
Cash	Owner receives cash, selects Cash, confirms
UPI / QR code	Customer scans the shop's QR code on their phone
PhonePe	Customer pays via PhonePe — owner confirms receipt
Google Pay	Customer pays via Google Pay — owner confirms receipt
Card / swipe machine	Customer swipes card on external POS machine — owner marks as Card

Important: For UPI, PhonePe, and Google Pay, the shop owner confirms payment manually after seeing the payment notification on their phone. The system does not automatically detect the payment — the owner presses "Confirm Payment" once they see the money has arrived.

After confirmation, the system:

- Saves the bill to the database with a unique bill number (e.g., BILL-2026-00847)
- Reduces stock count for each product sold
- Updates today's dashboard totals
- Generates the bill for printing

Feature 7 — Bill Generation and Printing

How the Customer Gets the Bill

Primary method — Thermal printer at the store:

- The system sends the bill to a thermal receipt printer connected to the shop owner's device (via Bluetooth or USB)
- A small receipt is printed instantly — same as you get at any kirana shop or supermarket
- The customer takes the physical printed receipt and leaves
- No phone number needed, no WhatsApp, no internet required for the customer

Secondary method — WhatsApp (only if phone number was provided):

- If the customer voluntarily gave their phone number, Ravi can optionally tap "Send on WhatsApp" to send a digital copy
- This is never automatic — it is always Ravi's manual choice
- Useful for customers who want a digital record

Tertiary method — PDF:

- Every bill is automatically saved as a PDF in Ravi's ShopEase account
- Ravi can find any past bill in Sales History
- If a customer comes back asking for a duplicate bill, Ravi can find and reprint it

What a Printed Bill Contains

```
      RAVI'S CLOTHING STORE
      Bangalore, Karnataka
      GSTIN: 29ABCDE1234F1Z5
      Ph: +91 98000 00000
-------------------------------------
Bill No : BILL-2026-00847
Date    : 25 Jun 2026, 12:34 PM
Customer: Arjun Kumar
-------------------------------------
Blue cotton shirt (L)       ₹1,200
Black slim jeans (32)
  MRP ₹1,800 → ₹1,500       ₹1,500
-------------------------------------
Subtotal                    ₹2,700
Discount                    - ₹300
GST (5%)                    + ₹135
-------------------------------------
TOTAL PAID                  ₹2,835
Payment: Cash
-------------------------------------
    Thank you for shopping!
  Visit again at Ravi's Clothing
-------------------------------------
```

Feature 8 — Email Reporting System

The system sends two types of automated emails to the shop owner. No human action required.

8.1 Daily Email Report

When: Every night at 9:00 PM automatically **To:** The email address the owner registered with **Subject line:** Daily Sales Report — [Date] | [Shop Name]

Contents of the daily email:

- Total sales revenue for the day (₹)
- Total profit for the day (₹)
- Number of bills generated
- Total items sold
- Full list of every product sold with quantity and revenue

- Payment mode breakdown (how much was cash, how much was UPI, etc.)
- Low stock alerts — products with stock below the threshold (default: 5 units)

8.2 Monthly Email Report

When: 1st of every month at 8:00 AM automatically (covering the previous full month) **To:** Same owner email **Subject line:** Monthly Report — [Month Year] | [Shop Name]
Attachment: Full PDF report of the month

Contents of the monthly email:

- Total monthly revenue
- Total monthly profit
- Total GST collected (useful for filing GST returns)
- Total discounts given
- Total number of bills generated
- Total items sold
- Average bill value
- Best-selling day of the month
- Top 5 products sold (by quantity)
- Payment mode breakdown for the month (% cash, % UPI, etc.)
- Inventory section:
  - Total stock purchased this month (units and ₹ value)
  - Total stock sold this month
  - Current stock remaining
  - Products that are out of stock
- Month vs previous month comparison:
  - Revenue up or down by %
  - Number of bills up or down
  - Average bill value up or down

8.3 Owner Controls for Email Settings

In the Settings page, the owner can:

- Change the email address reports are sent to
- Turn the daily report on or off
- Change the daily report send time (default: 9 PM)
- Turn the monthly report on or off
- Set the low stock alert threshold (default: 5 units)

- Choose whether to attach PDF to monthly email (default: on)

8.4 Tools Used for Email

Tool	Purpose	Why
Nodemailer	Sends emails from Node.js	Free, simple, works with Gmail SMTP
Gmail SMTP	Email delivery service	Free for small volume, reliable
node-cron	Schedules jobs at set times	Like an alarm clock for the server
PDFKit	Generates the monthly PDF attachment	Simple Node.js PDF library

How it works step by step:

1. node-cron fires at 9 PM every day (or 8 AM on the 1st of each month)
2. Server queries PostgreSQL for all relevant data (bills, items sold, profit, stock)
3. Data is filled into an HTML email template
4. Nodemailer sends the email to the owner's address via Gmail SMTP
5. System logs the send time. If it fails, it retries once after 10 minutes

---

Feature 9 — Product Management

The shop owner manages their product catalogue from the Products page:

- Add new product: name, category, barcode number, MRP, selling price, GST rate, stock quantity, product image (optional)
- Edit existing product details or price
- Delete a product
- Set minimum stock alert level per product
- View current stock levels across all products

Feature 10 — Sales History and Reports

- View all past bills in a list, newest first
- Search by customer name, bill number, or date range
- Filter by payment mode (cash only, UPI only, etc.)

- Click any bill to see the full breakdown
- Reprint any past bill
- Export bills to Excel for accountant/GST filing

Feature 11 — Coupon Management

The owner creates and manages all offers and coupons from this page:

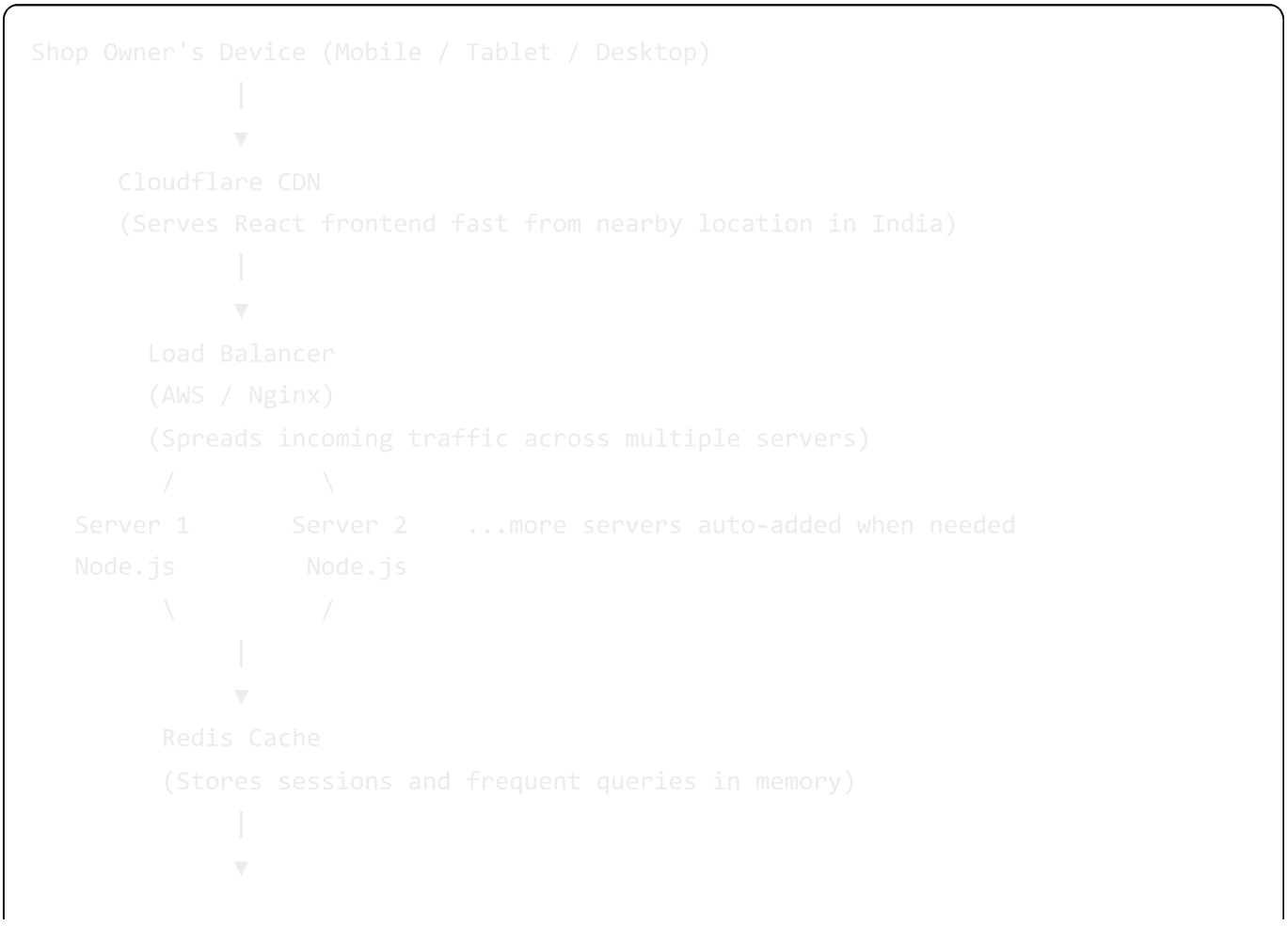
- Create a new coupon: code, type, value, expiry date, minimum purchase amount
- Enable or disable a coupon without deleting it
- Set which products a coupon applies to (all products or specific ones)
- See how many times each coupon has been used

---

4. System Design — How We Handle 1 Lakh Concurrent Users

This section explains the technical architecture that ensures ShopEase never crashes even when 1,00,000 shop owners are logged in and billing at the same time.

Architecture Overview



```
PostgreSQL Database
(Main permanent data store)
```

Why Each Component Is Needed

Problem	Solution	Why It Works
One server overloads with 1 lakh users	Load balancer splits traffic across multiple servers	Each server handles only a fraction of users
Every login hits the database — too slow	Redis caches the JWT session in memory	Memory access is 1000× faster than database queries
Users across India experience slow load	Cloudflare CDN serves frontend files from Mumbai, Delhi, Chennai nodes	Files come from nearby, not from one faraway server
Server goes down mid-billing	Multiple servers run in parallel — if one fails, others continue	No single point of failure
Database overloaded by product search queries	Redis caches the product list per shop	Repeated searches hit cache, not database
Traffic spikes during festivals (Diwali, etc.)	AWS auto-scaling adds servers automatically when load increases	Handles 5× normal traffic without manual intervention

---

5. Tech Stack — All Tools and Why

Frontend (What the User Sees and Interacts With)

Tool	What It Is	Why We Use It
React.js	JavaScript library for building user interfaces	Makes the billing screen update in real time (when you add a product, the total updates instantly without reloading the page). Used by Swiggy, Zomato, Flipkart.
Tailwind CSS	CSS utility framework	Makes mobile-responsive design fast. The billing app must work perfectly on small Android phones.
React Router	Page navigation library	Moves between pages (dashboard → new bill → billing screen) without full page reload — feels like a native app.

Tool	What It Is	Why We Use It
Axios	HTTP request library	Cleanly makes API calls from React to the Node.js backend (fetch product list, save bill, etc.).
Quagga.js	Barcode scanning in browser	Opens the phone camera and reads barcodes. No app installation needed — works in Chrome browser.
Web Speech API	Voice recognition	Built into Chrome and most Android browsers. No extra library needed. Converts spoken product names to text for search.
React-to-Print	Thermal printer connection	Sends the bill layout to the connected thermal printer with one click.

Why React and not plain HTML? A billing screen has many things changing at once — cart items, prices updating when you negotiate, GST recalculating, coupon applying. React manages all of this automatically through its state system. With plain HTML, you would need to manually update every element every time something changes, which quickly becomes unmanageable.

Backend (The Server — Handles All Business Logic)

Tool	What It Is	Why We Use It
Node.js	JavaScript runtime that runs on the server	You already know JavaScript. Node.js lets you use the same language for both frontend and backend. Also handles thousands of simultaneous requests well.
Express.js	Web framework for Node.js	Makes it simple to create API routes: /login, /products, /bills, /reports. Without it, you would write a lot more boilerplate code.
JWT (JSON Web Token)	Authentication token standard	After login, the user gets a token. Every request sends this token to prove identity. Stateless — the server does not store sessions in memory, so it scales perfectly across multiple servers.
bcrypt	Password hashing library	Encrypts passwords before storing. Even if the database is stolen, passwords cannot be read.
Redis	In-memory data store	Stores JWT sessions and cached product lists. 100× faster than PostgreSQL for repeated lookups. Critical for handling 1 lakh users.

Tool	What It Is	Why We Use It
Nodemailer	Email sending library	Sends daily and monthly reports to shop owner's email via Gmail SMTP.
node-cron	Task scheduler	Runs the daily report job at 9 PM and monthly report job on the 1st. Like an alarm clock built into the server.
PDFKit	PDF generation library	Creates the monthly report PDF that is attached to the monthly email. Also generates bill PDFs for storage.
MSG91 or Twilio	SMS gateway for India	Sends OTP to the shop owner's mobile number during login. MSG91 is India-specific and cheaper for Indian numbers.

Why Node.js and not Python (Django)? Both are valid choices. Node.js is chosen here because:

1. You already know JavaScript — one language for everything
2. Node.js handles many simultaneous connections better due to its non-blocking I/O model
3. The same developer can work on both frontend and backend without switching languages

Database

Tool	What It Is	Why We Use It
PostgreSQL	Relational database	Stores all permanent data: shops, products, customers, bills, coupons, stock. Relational means data is connected — a bill is linked to items, items are linked to products. This prevents corrupted data.
Redis	In-memory key-value store	Fast temporary storage for sessions and cache. Data here is not permanent — it expires automatically.

Why PostgreSQL and not MongoDB? Billing data has strict relationships:

- A bill contains many bill items
- Each bill item references a product
- A product belongs to a shop

Payments

Tool	What It Is	Why We Use It
Razorpay	Indian payment gateway	Supports UPI, PhonePe, Google Pay, cards, and net banking natively. Built for India. Cheaper than Stripe for Indian transactions.

Note: For the basic version (cash and UPI via QR code), Razorpay integration is optional. The owner can manually confirm UPI payments after seeing them on their phone. Razorpay becomes essential only when you want to track digital payments automatically.

Deployment (Making It Live on the Internet)

Tool	What It Is	Why We Use It
Vercel	Frontend hosting	Deploys the React app. Free tier available. Auto-deploys when you push code to GitHub.
Railway or Render	Backend hosting	Deploys the Node.js server. Simple setup with free tier for development.
Supabase	Managed PostgreSQL	Hosted PostgreSQL database — no server to manage. Free tier available.
Cloudflare	CDN and security	Makes the site fast across India by caching files at Indian data centres. Also protects from attacks. Free plan available.
GitHub	Code version control	Stores all code. Enables auto-deployments to Vercel and Railway. Essential for collaboration and backup.

6. Database Design

Table: shops

```
id              UUID, primary key
owner_name      VARCHAR — full name of the shop owner
shop_name       VARCHAR — displayed on bills and dashboard
```

```
phone           VARCHAR — used for OTP login
email           VARCHAR — where reports are sent
address         TEXT — printed on bills
gstin           VARCHAR — GST registration number
logo_url        VARCHAR — URL of shop logo image
report_email    VARCHAR — can be different from login email
daily_report    BOOLEAN — daily email on or off (default: true)
daily_time      TIME — when to send daily report (default: 21:00)
monthly_report  BOOLEAN — monthly email on or off (default: true)
low_stock_limit INTEGER — alert when stock below this (default: 5)
created_at      TIMESTAMP
```

Table: products

```
id              UUID, primary key
shop_id         UUID, foreign key → shops.id
name            VARCHAR
category        VARCHAR (clothing, footwear, grocery, electronics, etc.)
barcode         VARCHAR — scanned by Quagga.js
mrp             DECIMAL — original price
selling_price   DECIMAL — default selling price
cost_price      DECIMAL — what the owner paid (used for profit calculation)
gst_rate        DECIMAL — percentage (0, 5, 12, 18, 28)
stock_qty       INTEGER — current stock
min_stock_alert INTEGER — alert threshold per product
image_url       VARCHAR
is_active       BOOLEAN
created_at      TIMESTAMP
```

Table: customers

```
id              UUID, primary key
shop_id         UUID, foreign key → shops.id
name            VARCHAR — required
phone           VARCHAR — optional, null if not given
total_bills     INTEGER — how many times this customer has shopped
total_spent     DECIMAL — lifetime spend
created_at      TIMESTAMP
```

Table: bills

```
id              UUID, primary key
```

```
bill_number     VARCHAR — human readable, e.g. BILL-2026-00847
shop_id         UUID, foreign key → shops.id
customer_id     UUID, foreign key → customers.id (nullable)
customer_name   VARCHAR — stored directly in case customer record not created
subtotal        DECIMAL — before discount and tax
discount_amount DECIMAL — total discount given
tax_amount      DECIMAL — GST amount
total           DECIMAL — final amount paid
payment_mode    VARCHAR (cash, upi, phonepe, googlepay, card)
coupon_code     VARCHAR — if a coupon was used
pdf_url         VARCHAR — URL of the saved PDF bill
status          VARCHAR (completed, refunded)
created_at      TIMESTAMP
```

Table: bill_items

```
id              UUID, primary key
bill_id         UUID, foreign key → bills.id
product_id      UUID, foreign key → products.id
product_name    VARCHAR — stored at time of billing (in case product is deleted
later)
quantity        INTEGER
mrp             DECIMAL — at time of billing
selling_price   DECIMAL — after negotiation
discount_amount DECIMAL — on this item
gst_rate        DECIMAL
item_total      DECIMAL — selling_price × quantity
```

Table: coupons

```
id              UUID, primary key
shop_id         UUID, foreign key → shops.id
code            VARCHAR — what the owner types at billing
type            VARCHAR (percentage, flat, bogo, combo, minimum_purchase)
value           DECIMAL — discount amount or percentage
min_purchase    DECIMAL — minimum bill value to apply coupon
product_id      UUID — nullable, if coupon applies to specific product only
combo_product_id UUID — for combo offers (buy X get Y)
expiry_date     DATE
times_used      INTEGER — how many times this coupon has been applied
is_active       BOOLEAN
created_at      TIMESTAMP
```

Table: stock_purchases

```
id              UUID, primary key
shop_id         UUID, foreign key → shops.id
product_id      UUID, foreign key → products.id
quantity        INTEGER — units purchased from supplier
purchase_price  DECIMAL — per unit cost
total_cost      DECIMAL
supplier_name   VARCHAR
purchased_at    TIMESTAMP
```

Table: email_logs

```
id              UUID, primary key
shop_id         UUID, foreign key → shops.id
type            VARCHAR (daily, monthly)
sent_at         TIMESTAMP
status          VARCHAR (sent, failed)
retry_count     INTEGER
```

---

7. API Endpoints

Authentication

Method	Endpoint	Description
POST	/api/auth/send-otp	Sends OTP to shop owner's mobile via MSG91
POST	/api/auth/verify-otp	Verifies OTP, returns JWT token
POST	/api/auth/login	Email + password login (alternative)
POST	/api/auth/logout	Invalidates session in Redis

Dashboard

Method	Endpoint	Description
GET	/api/dashboard	Today's sales, profit, bill count, low stock alerts

Products

Method	Endpoint	Description
GET	/api/products	All products for this shop
POST	/api/products	Add a new product
PUT	/api/products/:id	Edit a product
DELETE	/api/products/:id	Delete a product
GET	/api/products/barcode/:code	Find product by barcode
GET	/api/products/search?q=	Search products by name

Customers

Method	Endpoint	Description
GET	/api/customers?phone=	Find customer by phone number
POST	/api/customers	Create a new customer record

Bills

Method	Endpoint	Description
POST	/api/bills	Create a new bill (saves to database, generates PDF)
GET	/api/bills	List all past bills (paginated)
GET	/api/bills/:id	Get a single bill with all items
GET	/api/bills/:id/pdf	Download the bill PDF
GET	/api/bills/search?q=	Search bills by customer name or bill number

Coupons

Method	Endpoint	Description
GET	/api/coupons	List all coupons

Method	Endpoint	Description
POST	/api/coupons	Create a new coupon
PUT	/api/coupons/:id	Edit a coupon
DELETE	/api/coupons/:id	Delete a coupon
POST	/api/coupons/validate	Validate a coupon code and return discount details

Reports and Email

Method	Endpoint	Description
GET	/api/reports/daily?date=	Get daily report data for any date
GET	/api/reports/monthly?month=	Get monthly report data
POST	/api/reports/send-daily	Manually trigger the daily email (for testing)
POST	/api/reports/send-monthly	Manually trigger the monthly email (for testing)

Settings

Method	Endpoint	Description
GET	/api/settings	Get shop settings
PUT	/api/settings	Update shop info, email preferences, logo

8. Email Reporting System

8.1 Daily Report — Triggered by node-cron at 9:00 PM

```javascript
```

```
// Runs every day at 9:00 PM
cron.schedule('0 21 * * *', async () => {
  const shops = await getAllShopsWithDailyReportEnabled();
  for (const shop of shops) {
    const data = await getDailyReportData(shop.id, today);
    const html = buildDailyEmailHTML(data);
    await sendEmail(shop.report_email, 'Daily Sales Report', html);
    await logEmailSent(shop.id, 'daily');
  }
});
```

Data pulled from database:

- SUM of bills.total WHERE date = today AND shop_id = shop
- COUNT of bills WHERE date = today
- SUM of (selling_price − cost_price) × quantity for profit
- GROUP BY payment_mode for payment breakdown
- All bill_items with product names and quantities
- Products where stock_qty < min_stock_alert

8.2 Monthly Report — Triggered by node-cron on 1st at 8:00 AM

javascript

```
// Runs at 8:00 AM on the 1st of every month
cron.schedule('0 8 1 * *', async () => {
  const shops = await getAllShopsWithMonthlyReportEnabled();
  for (const shop of shops) {
    const data = await getMonthlyReportData(shop.id, lastMonth);
    const html = buildMonthlyEmailHTML(data);
    const pdf = await generateMonthlyPDF(data);
    await sendEmail(shop.report_email, 'Monthly Report', html, pdf);
    await logEmailSent(shop.id, 'monthly');
  }
});
```

Additional data pulled for monthly:

- Week-by-week sales breakdown
- Top 5 products by quantity sold
- Stock purchases vs stock sold
- Comparison with previous month figures

- Total GST collected (for GST filing)

---

9. UI Screens

Screen	Purpose	Key Elements
Login	Owner authentication	Phone + OTP, or email + password
Dashboard	Daily overview	Sales total, profit, bills count, stock alerts, New Bill button
New Bill — Customer	Capture customer info	Name field (required), phone field (optional and clearly labelled so)
New Bill — Products	Add items to cart	Barcode scan, voice button, search bar, cart with + / − quantity
Billing screen	Review and finalise	Item list with MRP and negotiated price, discount, coupon field, GST, total, payment mode selection
Payment confirmed	Confirmation	Success screen, Print button, optional WhatsApp send button
Products	Manage catalogue	List, Add, Edit, Delete products
Coupon management	Create offers	Add/edit/disable coupons and offers
Sales history	View past bills	Searchable list, date filter, click to view, reprint option
Reports	Charts	Daily/monthly charts, top products bar chart
Settings	Shop configuration	Shop info, logo, GSTIN, email preferences, report schedule, stock alert threshold

---

10. Build Order — Phase by Phase

Phase 1 — Foundation (Weeks 1 to 3)

Goal: Get a working login and basic dashboard running.

1. Set up React project with Tailwind CSS

2. Set up Node.js and Express backend
3. Connect PostgreSQL database and create all tables
4. Build the shops table — owner registration
5. Implement OTP login: MSG91 integration + JWT + Redis session
6. Build the dashboard screen (static data first, then wire to API)
7. Set up GitHub repository and connect to Vercel (frontend) and Railway (backend)

Phase 2 — Core Billing (Weeks 4 to 6)

Goal: Complete a full billing transaction from start to print.

8. Build product management screen (add, edit, delete products)
9. Build the new bill — customer details screen
10. Build product search and manual selection
11. Build the cart with quantity controls
12. Build the billing screen (price, GST, totals)
13. Implement price negotiation (edit price per item)
14. Save bill to database (POST /api/bills)
15. Generate PDF bill using PDFKit
16. Connect to thermal printer using React-to-Print

Phase 3 — Advanced Features (Weeks 7 to 9)

Goal: Add barcode, voice, coupons, and payment modes.

17. Barcode scanning with Quagga.js
18. Voice input with Web Speech API
19. Coupon validation system
20. Offer types — BOGO, combo, percentage, flat
21. Payment mode selection screen
22. Sales history screen with search and filter
23. Reprint any past bill

Phase 4 — Email Reports (Weeks 10 to 11)

Goal: Automated daily and monthly email reporting.

24. Set up Nodemailer with Gmail SMTP
25. Build the daily report HTML email template
26. Build node-cron job for 9 PM daily trigger

27. Write the daily data query (sales, profit, stock alerts)
28. Build the monthly report HTML email template
29. Build PDFKit monthly PDF generator
30. Build node-cron job for 1st of month at 8 AM
31. Build the email settings page for owners
32. Test email delivery and retry logic

Phase 5 — Polish and Launch (Week 12)

Goal: Secure, fast, and live.

33. Add Redis caching for product lists and sessions
34. Set up Cloudflare CDN
35. Security audit — HTTPS, rate limiting, input validation
36. Mobile testing on Android (primary device for Indian shop owners)
37. Buy domain (e.g., shopease.in)
38. Final deployment and go live

---

11. Non-Functional Requirements

Requirement	Target
Concurrent users	1,00,000 simultaneous without downtime
Page load time	Under 2 seconds on 4G mobile
Bill generation time	Under 3 seconds from confirm to PDF ready
Email delivery	Daily report delivered within 2 minutes of 9 PM trigger
Uptime	99.9% (less than 9 hours downtime per year)
Device support	Android phones and tablets (primary), iPhone, Windows laptop
Browser support	Chrome (primary), Safari, Firefox
Thermal printer	Works with any ESC/POS compatible printer (most Indian shop printers)
Offline capability	Product list cached locally — barcode scan works without internet
Data backup	PostgreSQL backed up daily to AWS S3

Requirement	Target
Security	HTTPS, JWT, bcrypt passwords, rate limiting on OTP endpoint

12. What You Will Learn by Building This

Feature	Skills Gained
OTP login	Node.js, Express, JWT, Redis, third-party SMS API (MSG91)
Barcode scan	Quagga.js, browser camera API, React hooks
Voice input	Web Speech API, React state management
Product search	PostgreSQL LIKE queries, REST API design
Cart and billing	JavaScript calculations, React state, complex UI logic
Price negotiation	Business logic, real-time recalculation in React
GST calculation	Business rules in code, conditional logic
Coupon system	Complex database queries, discount rules
PDF generation	PDFKit, server-side file generation
Thermal printing	ESC/POS protocol, React-to-Print library
Daily email	Nodemailer, Gmail SMTP, HTML email templates
Monthly email	node-cron, scheduler design, PDF attachment
Data aggregation	Complex SQL queries — SUM, GROUP BY, JOIN, date filters
Redis caching	Cache design, session management, expiry
Load balancing	System design, horizontal scaling, Nginx
Deployment	Vercel, Railway, Supabase, Cloudflare, GitHub CI/CD

Appendix — Third Party Services Summary

Service	Purpose	Cost
MSG91	OTP SMS to Indian numbers	Pay per SMS (~₹0.15 per OTP)
Gmail SMTP	Sending report emails	Free up to 500 emails/day
Vercel	Frontend hosting	Free for small projects
Railway	Backend hosting	Free tier then ~$5/month
Supabase	PostgreSQL database	Free tier available
Cloudflare	CDN and security	Free plan available
Razorpay	Payments (future)	2% per transaction

End of PRD — ShopEase Billing System v2.0 Next step: Begin Phase 1 — set up the React project and Node.js backend